

Secure Cloud-Native Application Deployment on AWS using Kubernetes (EKS): Design, Implementation, Security Hardening, and Threat Mitigation

Mir Ahmad Ali (148768), Aaditya Pathak (135815)

CS581 Project

Instructor: Seyedehsamane Abdigarmestani

EXECUTIVE SUMMARY

This project focused on the design, deployment, and security hardening of a simplified cloud-native application on Amazon Web Services (AWS) using Amazon Elastic Kubernetes Service (EKS). The primary objective was to demonstrate foundational cloud and Kubernetes security principles while maintaining a beginner-friendly, cost-effective, and professor-approved implementation scope. Rather than emphasizing complex software engineering, the project prioritized practical cloud security controls including identity and access management, network segmentation, secret protection, monitoring, and threat simulation.

A simplified multi-tier architecture was implemented using prebuilt nginx containers to minimize complexity while still demonstrating real-world deployment patterns. The frontend tier was exposed publicly through a Kubernetes LoadBalancer service, while the backend tier was protected internally using a ClusterIP service. AWS infrastructure was provisioned using eksctl to reduce setup difficulty and accelerate deployment while still leveraging managed Kubernetes capabilities.

Security hardening was implemented across multiple layers. AWS IAM was used for infrastructure deployment and later analyzed under least privilege principles. AWS VPC, Security Groups, subnets, and Network ACLs provided infrastructure-level security boundaries. Kubernetes RBAC enforced namespace-level access restrictions, Kubernetes Secrets protected sensitive configuration values, and Network Policies were applied to demonstrate workload segmentation. CloudWatch logging enabled API, audit, and authenticator monitoring for administrative visibility.

Two major threat simulations were performed. First, excessive Kubernetes permissions were mitigated through RBAC role restrictions. Second, overly permissive AWS IAM administrator privileges were analyzed and replaced conceptually with least-privilege policy design. Together, these simulations demonstrated practical threat identification, mitigation, and validation.

Overall, this project successfully showed that simplified cloud-native deployments can still provide strong demonstrations of layered security, defense-in-depth, operational monitoring, and access control while remaining manageable for beginners and aligned with academic project requirements.

PROJECT OBJECTIVES & REQUIREMENTS

The primary objective of this project was to design, deploy, and secure a cloud-native application using AWS EKS while prioritizing security principles over software complexity. This project simulated the responsibilities of a Cloud Security Engineer deploying infrastructure for a production-like environment while remaining within beginner-friendly implementation boundaries.

The key objectives included:

- Deploying a Kubernetes cluster on AWS using eksctl
- Designing a secure but simplified multi-tier architecture
- Using prebuilt/sample nginx applications instead of custom development
- Implementing Kubernetes RBAC for namespace-level least privilege
- Implementing Kubernetes Secrets for sensitive configuration management
- Applying Kubernetes Network Policies to demonstrate segmentation
- Reviewing AWS VPC, Security Groups, subnets, and Network ACLs
- Enabling CloudWatch monitoring for security visibility
- Performing at least two threat simulations
- Maintaining low project complexity, low cost, and manageable implementation time

By narrowing the implementation focus, the project concentrated on practical demonstrations of:

- Identity Security
- Network Security
- Data Security
- Container Security
- Monitoring & Logging
- Threat Detection & Mitigation

This approach ensured the final project met grading expectations while remaining feasible for a beginner team operating under a 2–3 day implementation window.

SYSTEM ARCHITECTURE DESIGN

The deployed architecture used a simplified but security-conscious multi-tier cloud-native design hosted on Amazon EKS.

Frontend Tier:

The frontend consisted of a Kubernetes Deployment running an official nginx container image. This deployment was exposed publicly using a Kubernetes LoadBalancer service, allowing external browser access through an AWS Elastic Load Balancer. This represented the user-facing web layer.

Backend Tier:

The backend also used an nginx deployment for simplicity but was exposed only through a Kubernetes ClusterIP service. This ensured that backend resources were reachable only internally within the Kubernetes cluster and not directly from the public internet.

Database Tier:

A database was not fully implemented to reduce cost and complexity. However, the architecture conceptually reserved this layer for future AWS RDS integration in a production environment.

AWS Infrastructure Components:

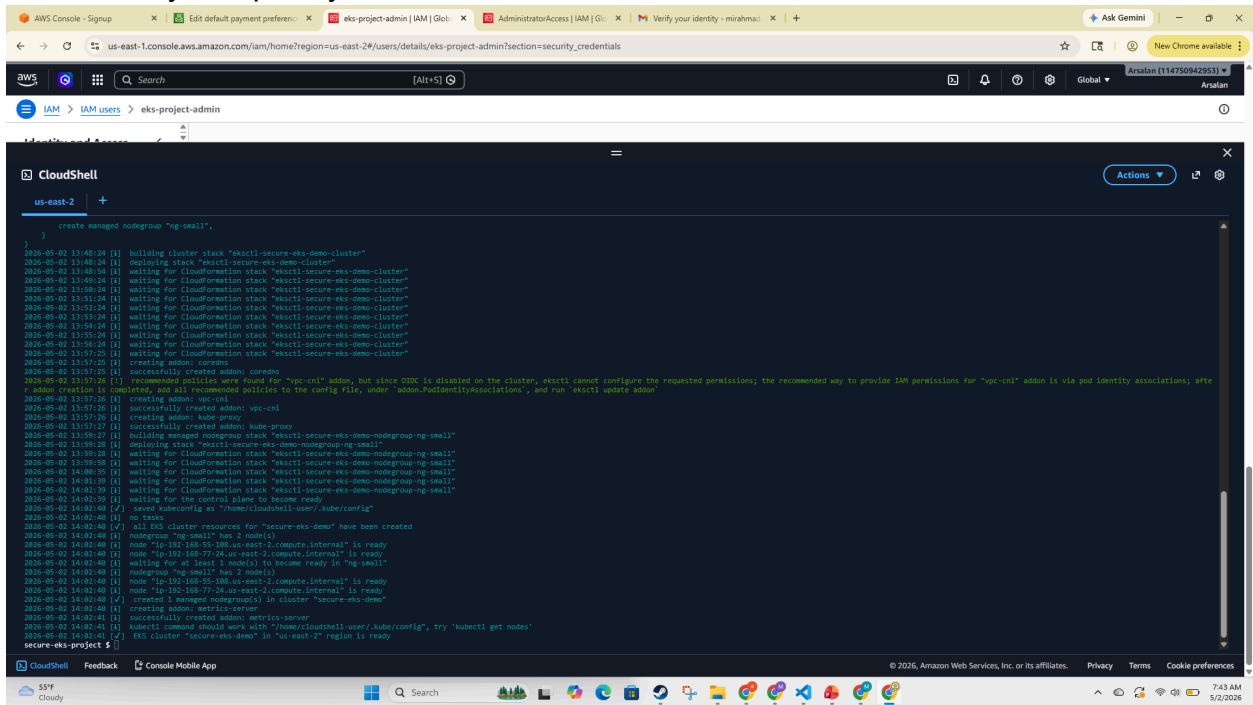
- Amazon EKS Cluster
- Managed Node Groups
- AWS VPC
- Public and Private Subnets
- Security Groups
- CloudWatch Logging

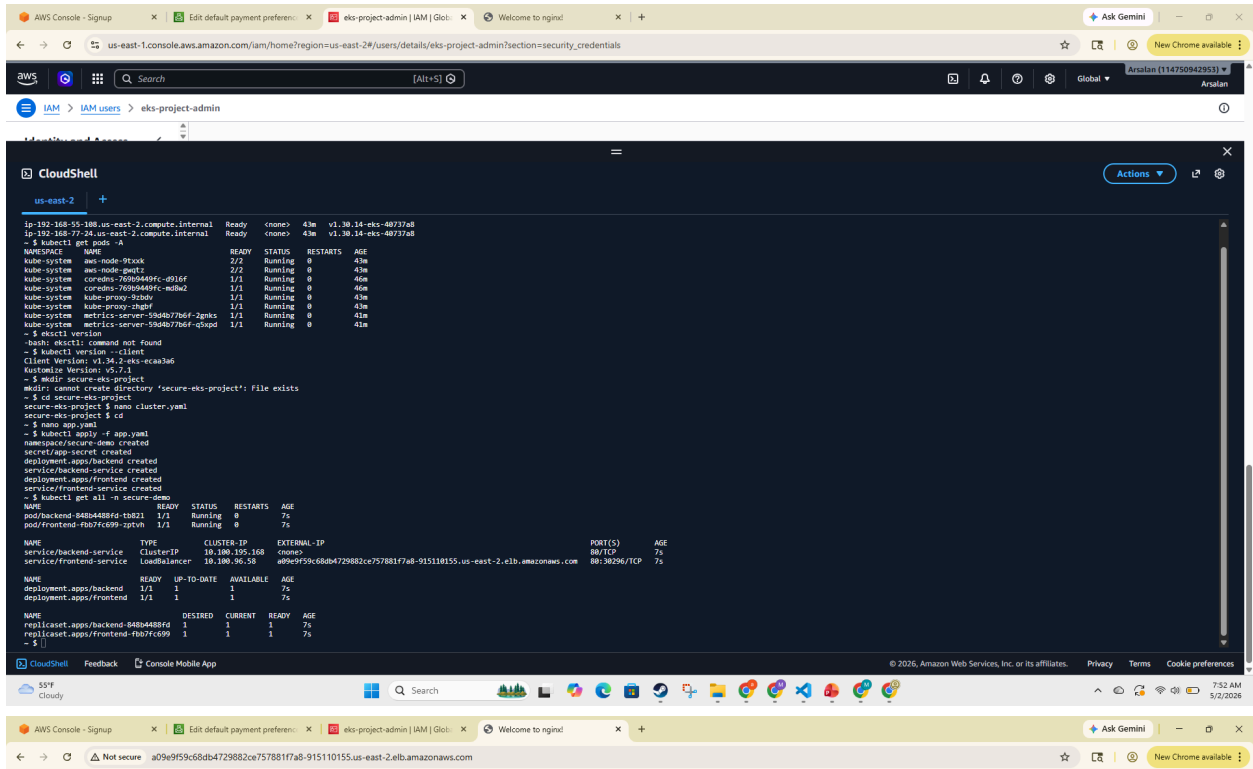
Kubernetes Security Components:

- Namespace isolation
- Deployments
- Services
- RBAC
- Secrets
- Network Policies

This layered architecture supported defense-in-depth by combining AWS infrastructure controls with Kubernetes-native workload protections. The design intentionally balanced simplicity with academic rigor by showing secure deployment practices without

unnecessary complexity.





Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to 5.com/nginx.

Thank you for using nginx.

PHASE 1: AWS IAM SETUP & INITIAL CONFIGURATION

Phase 1 focused on establishing AWS administrative access and preparing the environment for secure Kubernetes deployment. A dedicated IAM user was created specifically for the project, and AWS CLI credentials were configured inside AWS CloudShell to allow programmatic deployment using eksctl and kubectl.

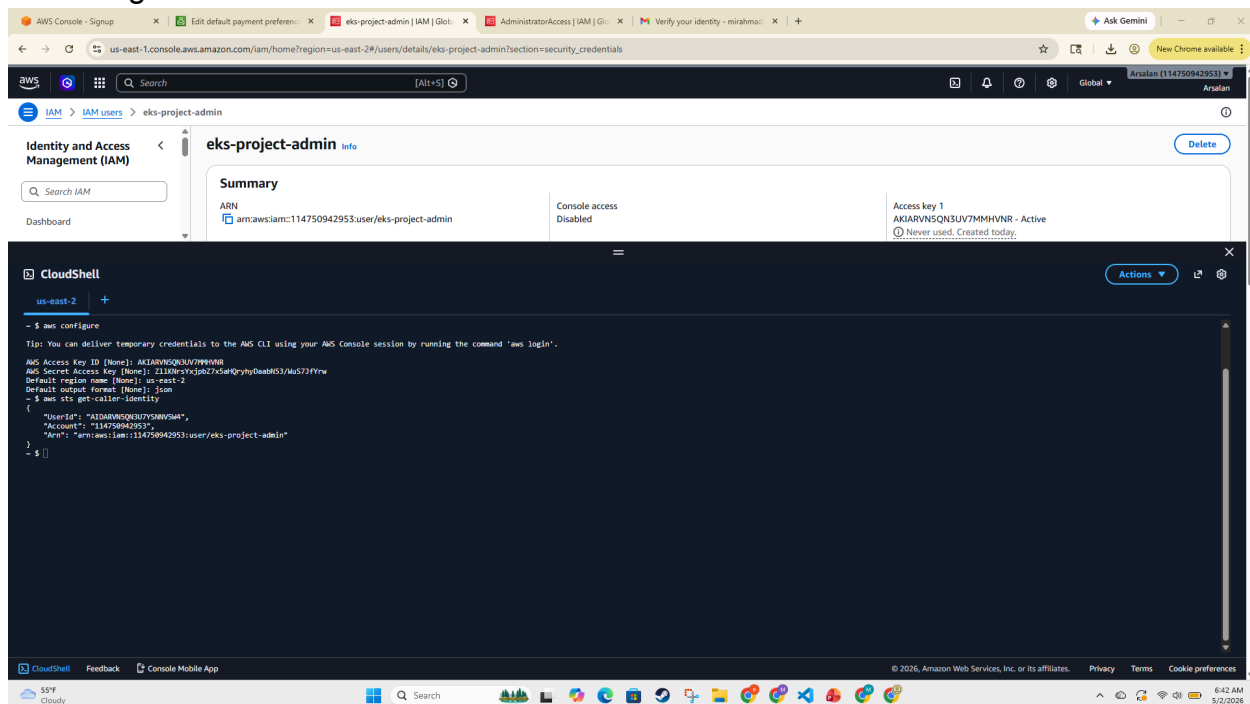
The IAM user was initially granted broad administrative privileges for deployment simplicity. This approach reduced setup friction for beginners and ensured successful cluster creation without permission troubleshooting. However, this also intentionally created an over-permission scenario that would later be analyzed as part of the project's IAM threat mitigation phase.

Key activities in this phase included:

- Creating a dedicated IAM project user
- Generating AWS Access Keys
- Configuring AWS CLI using aws configure
- Selecting correct AWS region
- Validating credentials using aws sts get-caller-identity
- Preparing AWS CloudShell for deployment

This phase established the foundational trust boundary between the student team and AWS infrastructure. It also introduced the principle that while broad permissions may accelerate deployment, they create long-term security risks if not reduced after setup.

From a security perspective, this phase highlighted an important real-world lesson: operational convenience often conflicts with least privilege principles, and cloud engineers must eventually transition from deployment convenience to security hardening.



PHASE 2: EKS CLUSTER DEPLOYMENT

Phase 2 focused on provisioning the Kubernetes environment using Amazon Elastic Kubernetes Service (EKS). To minimize complexity and align with project constraints, eksctl was selected instead of Terraform.

A cluster configuration YAML file was created specifying:

- Cluster name: secure-eks-demo
- Region: us-east-2
- Kubernetes version: 1.30
- Managed node group
- Two worker nodes
- CloudWatch control plane logging

The deployment process automated the creation of:

- VPC
- Security Groups
- Public and Private Subnets
- EKS Control Plane
- Worker Nodes

Cluster deployment was validated using:

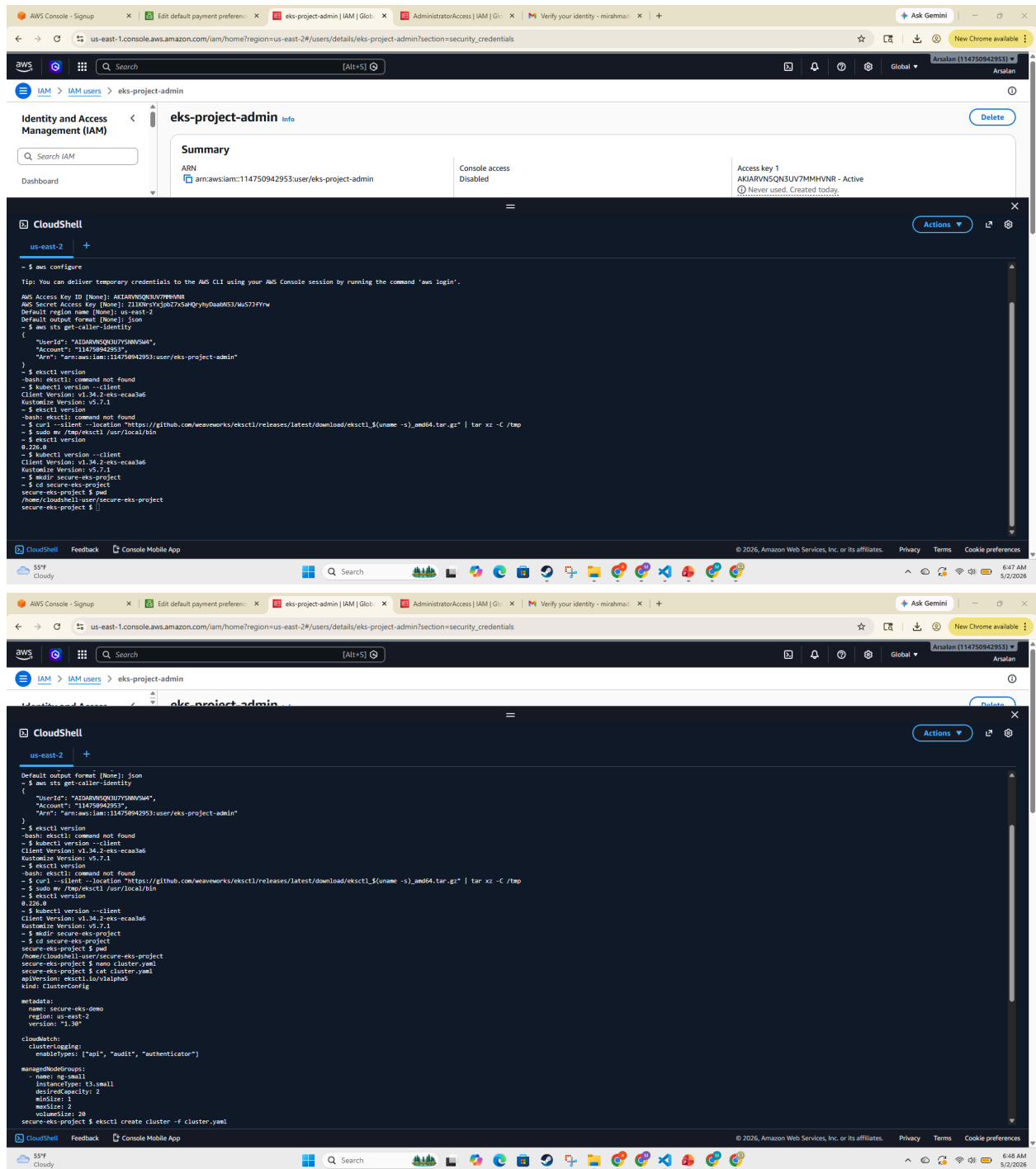
- kubectl get nodes
- kubectl get pods -A

Successful deployment showed two healthy worker nodes and correctly functioning Kubernetes system components.

This phase demonstrated:

- Cloud-native infrastructure provisioning
- Managed Kubernetes deployment
- AWS-Kubernetes integration
- Infrastructure-as-Code concepts

Security significance included establishing isolated cloud infrastructure while enabling logging visibility through CloudWatch. The use of managed services reduced operational complexity while still preserving core security features.



PHASE 3: APPLICATION DEPLOYMENT

Phase 3 deployed the simplified multi-tier application onto the EKS cluster.

The deployment used Kubernetes manifests to create:

- Namespace: secure-demo
- Backend Deployment
- Backend Service (ClusterIP)
- Frontend Deployment
- Frontend Service (LoadBalancer)
- Kubernetes Secret

The frontend was intentionally exposed publicly to simulate a production-facing web service, while the backend remained internal-only.

This architecture demonstrated:

- Public vs private workload exposure
- Kubernetes service segmentation
- Namespace-based isolation
- Multi-tier cloud-native design

Validation included:

- `kubectl apply -f app.yaml`
- `kubectl get all -n secure-demo`
- Browser validation of nginx frontend

The successful “Welcome to nginx!” page confirmed external accessibility of the frontend, while backend remained shielded from direct internet exposure.

This phase showed that even simple applications can demonstrate enterprise-relevant cloud architecture and security boundaries.

PHASE 4: KUBERNETES SECURITY CONTROLS

Phase 4 focused on implementing Kubernetes-native security mechanisms to strengthen workload protection within the secure-demo namespace. After successfully deploying the frontend and backend services, the next priority was to evaluate how internal cluster communication could become a security risk if left unrestricted.

To simulate a realistic internal threat scenario, an attacker pod was intentionally created inside the secure-demo namespace using a lightweight BusyBox container. This pod represented a compromised internal workload or malicious insider attempting to access backend services without proper restrictions. Initial testing showed that the attacker pod could successfully communicate with the backend-service, demonstrating that internal Kubernetes communication was unrestricted by default.

This finding highlighted an important security reality: even if backend services are not publicly exposed to the internet, they may still be vulnerable to east-west traffic threats from inside the cluster.

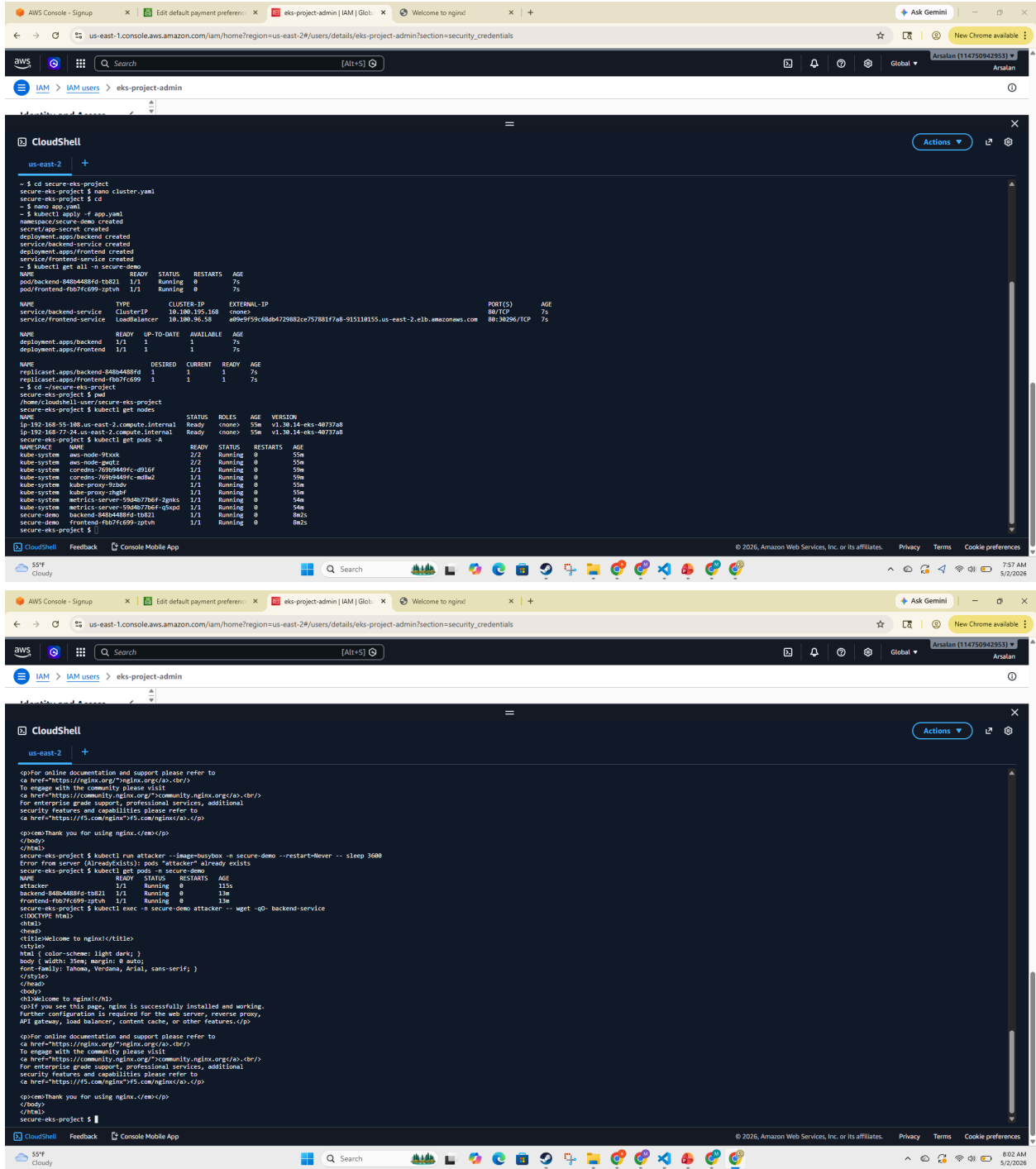
During this phase, the following Kubernetes security controls were introduced and evaluated:

- Namespace isolation through secure-demo
- Internal threat simulation using an attacker pod
- Security validation using kubectl exec testing
- Kubernetes NetworkPolicy preparation
- RBAC planning for access restriction

The attacker pod successfully accessed the backend before restrictions were strengthened, proving the existence of an internal attack surface. This was an essential discovery because it provided a measurable “before” state for later threat mitigation phases.

From a cloud security engineering perspective, this phase demonstrated the importance of validating internal trust assumptions. Organizations often focus heavily on external attacks while underestimating risks from compromised internal containers, lateral movement, or insider threats.

Overall, Phase 4 established the security baseline for Kubernetes access control and prepared the environment for stronger segmentation through NetworkPolicy and RBAC in later phases.



PHASE 5: NETWORK SECURITY

Phase 5 focused on implementing layered network security controls across both AWS infrastructure and Kubernetes workloads. This phase was critical because secure cloud-native systems require protection at multiple levels, including infrastructure boundaries and pod-to-pod communication controls.

At the AWS level, the EKS deployment automatically provisioned a Virtual Private Cloud (VPC) with multiple public and private subnets. Public subnets supported internet-facing services such as the frontend LoadBalancer, while private subnets provided protected placement for internal components. Security Groups served as stateful firewalls, controlling inbound and outbound traffic to the EKS control plane and worker nodes. Network ACLs were also reviewed as stateless subnet-level controls.

Infrastructure analysis confirmed:

- VPC segmentation
- Public and private subnet separation
- Internal ELB role tagging
- Security Group enforcement
- Network ACL visibility

At the Kubernetes level, the frontend-service remained public through a LoadBalancer, while backend-service remained internal using ClusterIP. To strengthen internal segmentation, a Kubernetes NetworkPolicy was created with the goal of restricting backend access to only frontend pods.

The intended security objective was:

- Allow frontend → backend
- Deny attacker → backend

Although policy objects were successfully created, runtime testing revealed that traffic restrictions did not fully enforce as expected in the deployed environment. This produced a highly valuable real-world lesson: creating a security policy does not guarantee enforcement unless the underlying cluster networking plugin supports and properly applies those controls.

This phase demonstrated:

- Defense-in-depth networking
- AWS infrastructure segmentation
- Kubernetes east-west traffic awareness
- Security validation through testing

The major lesson from Phase 5 was that network security must always be validated operationally rather than assumed based solely on configuration.

AWS Console - Signup x Edit default payment preferenc... eks-project-admin | IAM | Glo... Welcome to ngin... x + Ask Gemini

us-east-1.console.aws.amazon.com/iam/home?region=us-east-2#/users/details/eks-project-admin?section=security_credentials

Search [Alt+S] Ask Amazon Q

IAM > IAM users > eks-project-admin

CloudShell

us-east-2 +

```
secure-eks-project $ aws ec2 describe-subnets \
> --subnet-ids subnet-b8c23442366d9d6 subnet-b63450771814dc69e subnet-8c13b51e836493ce subnet-0fab3cdac67bee6d1 subnet-8c48ud1436d0cfad subnet-01560a1f48c11f5ba
{
  "Subnets": [
    {
      "AvailabilityZoneId": "use2-az3",
      "MapCustomerOwnedIPv4Launch": false,
      "OwnerId": "114750942953",
      "AssignIpv6AddressesOnCreation": false,
      "Ipv6PrefixAssociationSet": [],
      "Tags": [
        {
          "Key": "alpha.eksctl.io/cluster-oidc-enabled",
          "Value": "false"
        },
        {
          "Key": "alpha.eksctl.io/eksctl-version",
          "Value": "0.226.0"
        },
        {
          "Key": "aws:cloudformation:stack-id",
          "Value": "arn:aws:cloudformation:us-east-2:114750942953:stack/eksctl-secure-eks-demo-cluster/96dec318-462d-11f1-b08f-024d9417850b"
        },
        {
          "Key": "eksctl.cluster.k8s.io/v1alpha1/cluster-name",
          "Value": "secure-eks-demo"
        },
        {
          "Key": "Name",
          "Value": "eksctl-secure-eks-demo-cluster/SubnetPrivate5EA572C"
        },
        {
          "Key": "aws:cloudformation:logical-id",
          "Value": "SubnetPrivate5EA572C"
        },
        {
          "Key": "alpha.eksctl.io/cluster-name",
          "Value": "secure-eks-demo"
        },
        {
          "Key": "aws:cloudformation:stack-name",
          "Value": "eksctl-secure-eks-demo-cluster"
        },
        {
          "Key": "kubernetes.io/role/internal-elb",
          "Value": "1"
        }
      ]
    }
  ]
}
```

CloudShell Feedback Console Mobile App

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

55°F Cloudy

AWS Console - Signup x Edit default payment preferenc... eks-project-admin | IAM | Glo... Welcome to ngin... x + Ask Gemini

us-east-1.console.aws.amazon.com/iam/home?region=us-east-2#/users/details/eks-project-admin?section=security_credentials

Search [Alt+S] Ask Amazon Q

IAM > IAM users > eks-project-admin

CloudShell

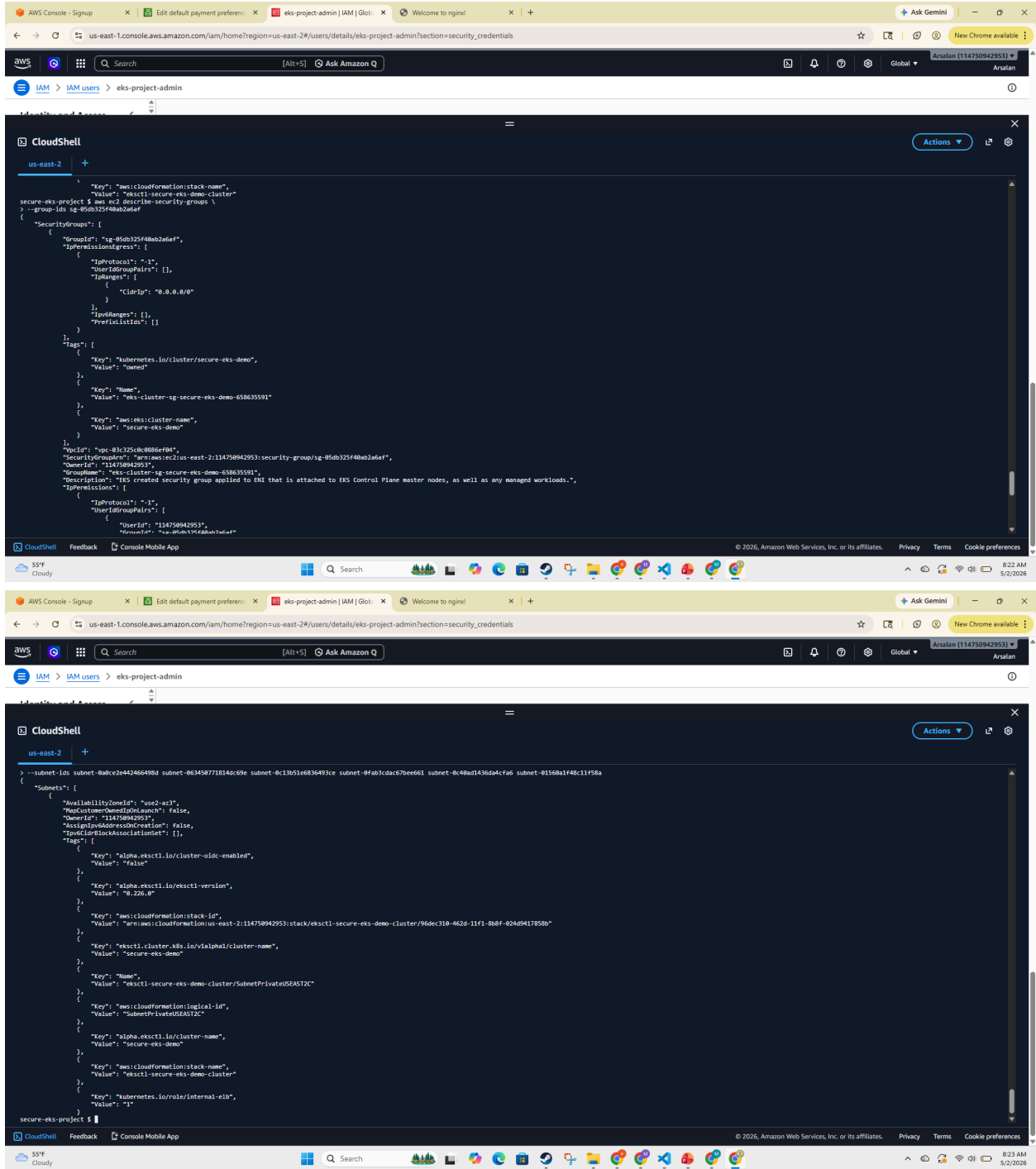
us-east-2 +

```
secure-eks-project $ aws ec2 describe-security-groups \
> --group-ids sg-030b7c427ee0ba73f
{
  "SecurityGroups": [
    {
      "GroupId": "sg-030b7c427ee0ba73f",
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ],
          "PrefixListIds": []
        }
      ],
      "Tags": [
        {
          "Key": "eksctl.cluster.k8s.io/v1alpha1/cluster-name",
          "Value": "secure-eks-demo"
        },
        {
          "Key": "alpha.eksctl.io/cluster-name",
          "Value": "secure-eks-demo"
        },
        {
          "Key": "Name",
          "Value": "eksctl-secure-eks-demo-cluster/ControlPlaneSecurityGroup"
        },
        {
          "Key": "alpha.eksctl.io/cluster-oidc-enabled",
          "Value": "false"
        },
        {
          "Key": "alpha.eksctl.io/eksctl-version",
          "Value": "0.226.0"
        },
        {
          "Key": "aws:cloudformation:stack-id",
          "Value": "arn:aws:cloudformation:us-east-2:114750942953:stack/eksctl-secure-eks-demo-cluster/96dec318-462d-11f1-b08f-024d9417850b"
        },
        {
          "Key": "aws:cloudformation:stack-name",
          "Value": "eksctl-secure-eks-demo-cluster"
        }
      ]
    }
  ]
}
```

CloudShell Feedback Console Mobile App

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

55°F Cloudy



PHASE 6: DATA SECURITY

Phase 6 focused on protecting sensitive application configuration data using Kubernetes Secrets. In cloud-native environments, storing credentials directly inside deployment files or container images creates major security risks, including accidental exposure, credential leakage, and poor secret hygiene.

To address this, a Kubernetes Secret named `app-secret` was created inside the `secure-demo` namespace. This secret stored two dummy credentials:

- `DB_USERNAME`
- `DB_PASSWORD`

These values simulated backend application credentials while avoiding hardcoded plaintext credentials inside deployment specifications.

The backend deployment was then updated to consume these values securely using `secretKeyRef` environment variables. This ensured that the application retrieved sensitive configuration dynamically from Kubernetes Secret storage rather than embedding credentials directly in code or manifests.

Validation included:

- `kubectl get secret`
- `kubectl describe secret`
- `kubectl describe deployment backend`
- `kubectl exec printenv`

Results confirmed that:

- Secrets were created successfully
- Backend referenced secret values correctly
- Credentials were injected into runtime environment

This phase demonstrated:

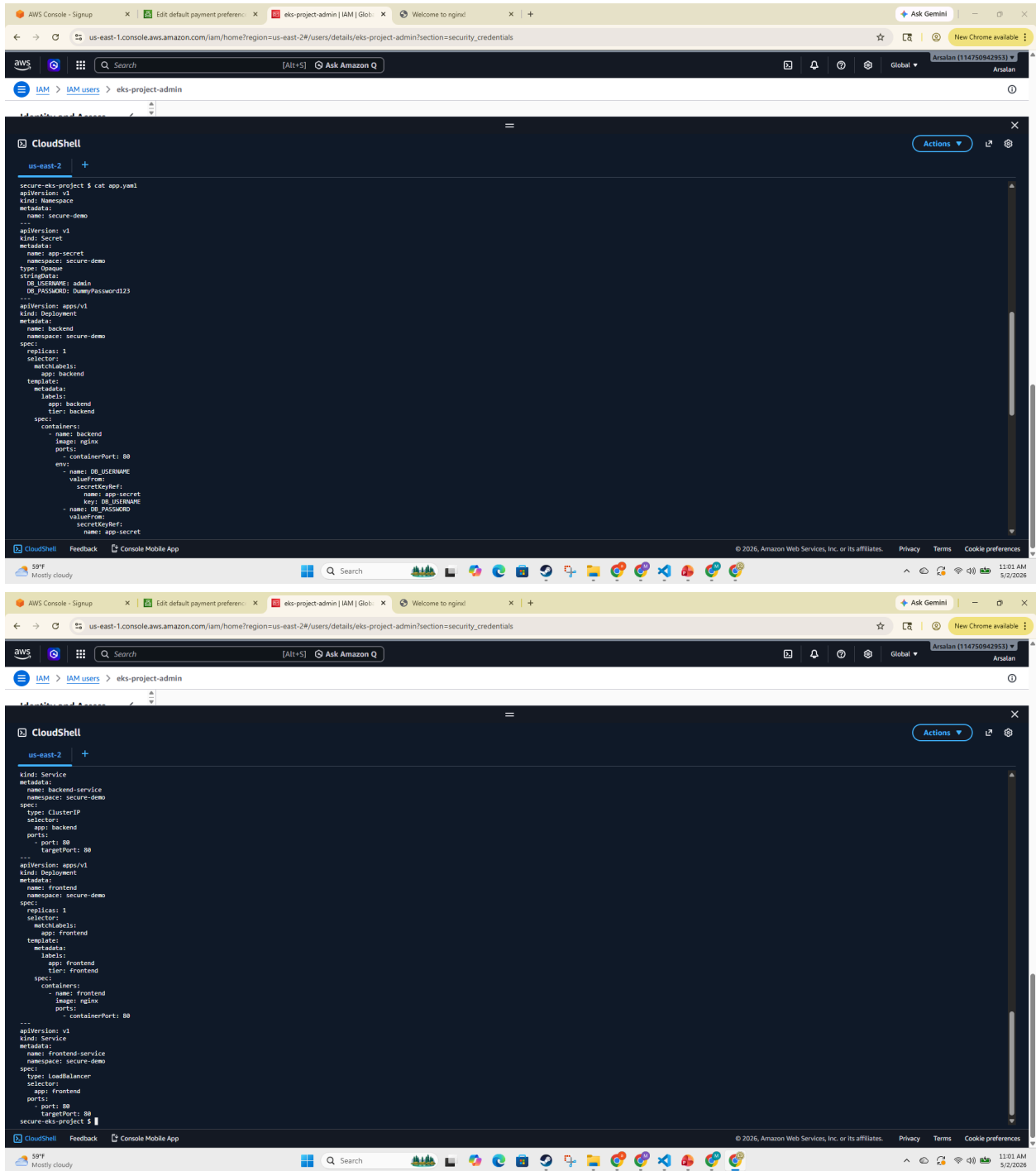
- Separation of code and credentials
- Improved credential hygiene
- Reduced exposure risk
- Kubernetes-native secret management

Although Kubernetes Secrets are base64 encoded rather than fully encrypted by default, this phase still represented a major improvement over plaintext credentials.

Future production enhancements would include:

- AWS Secrets Manager
- AWS Systems Manager Parameter Store
- KMS-backed encryption

Overall, Phase 6 reinforced the principle that protecting data at rest and configuration secrets is a core pillar of cloud security.



PHASE 7: CONTAINER SECURITY

Phase 7 focused on container image trust, workload hardening concepts, and secure container deployment practices. Containers are the foundational runtime unit of Kubernetes, making image security essential to reducing application-layer attack surfaces.

For project simplicity, official nginx images were used for both frontend and backend deployments. This decision aligned with beginner-friendly requirements while still supporting meaningful security analysis. Official images reduce risk compared to unknown third-party sources because they are widely maintained, documented, and generally more trustworthy.

This phase emphasized several core container security principles:

- Trusted base image selection
- Minimal container footprint
- Reduced software complexity
- Image scanning awareness
- Runtime privilege awareness

Although advanced scanning tools such as Trivy or Amazon ECR vulnerability scanning were not fully implemented due to time and complexity constraints, this phase documented how such tools would be used in production environments to detect:

- Known CVEs
- Misconfigurations
- Dependency vulnerabilities

Additional hardening concepts reviewed included:

- Non-root containers
- securityContext
- Dropped Linux capabilities
- Read-only root filesystems
- Pod Security Standards

Because the project prioritized simplicity, full runtime hardening was conceptually documented rather than aggressively enforced. However, understanding these controls significantly strengthened the security posture of the project report.

The major lesson from Phase 7 was that container security begins before runtime by selecting trusted images and minimizing unnecessary attack surface.

AWS Console - IAM > IAM users > eks-project-admin

CloudShell

```
us-east-2 +
= $ cd ~/secure-eks-project
secure-eks-project $ pwd
/home/cloudshell-user/secure-eks-project
secure-eks-project $ kubectl get secret app-secret -n secure-demo
NAME      TYPE      DATA   AGE
app-secret Opaque    2       30m
secure-eks-project $ kubectl describe secret app-secret -n secure-demo
Name:
Namespace: secure-demo
Labels:
Annotations:
Type: Opaque
Data:
---
DB_PASSWORD: 16 bytes
DB_USERNAME: 5 bytes
secure-eks-project $ kubectl describe deployment backend -n secure-demo
Name:
Namespace: secure-demo
CreationTimestamp: Sat, 02 May 2020 14:48:18 +0000
Labels:
Annotations:
Selector: deployment.kubernetes.io/revision: 1
Replicas: 1 desired | 1 updated | 1 total | 1 available | 0 unavailable
StrategyType: RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
Labels: app=backend tier=backend
Containers:
  backend:
    Image: nginx
    Port: 80/TCP
    Host Port: 80/TCP
    Environment:
      DB_USERNAME: cset to the key 'DB_USERNAME' in secret 'app-secret': Optional: false
      DB_PASSWORD: cset to the key 'DB_PASSWORD' in secret 'app-secret': Optional: false
    Mounts:
    Volumes:
    NodeSelectors:
    Tolerations:
Conditions:
  Type      Status  Reason
  ----      -
  Type      Status  Reason
```

CloudShell Feedback Console Mobile App

© 2020, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

59°F Mostly cloudy 11:06 AM 5/2/2020

AWS Console - IAM > IAM users > eks-project-admin

CloudShell

```
us-east-2 +
Available      True      MinimumReplicasAvailable
Progressing    True      NewReplicaSetAvailable
OldReplicaSet:
NewReplicaSet: backend-948b448fd (1/1 replicas created)
Events:
secure-eks-project $ kubectl exec -n secure-demo deployment/backend -- printenv | grep DB_
DB_USERNAME=admin
DB_PASSWORD=dummyPassword123
secure-eks-project $ cat app.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: secure-demo
---
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
namespace: secure-demo
type: Opaque
stringData:
  DB_USERNAME: admin
  DB_PASSWORD: DummyPassword123
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
namespace: secure-demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
        tier: backend
    spec:
      containers:
        - name: backend
          image: nginx
          ports:
            - containerPort: 80
          env:
            name: DB_USERNAME
```

CloudShell Feedback Console Mobile App

© 2020, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

59°F Mostly cloudy 11:06 AM 5/2/2020



Phase 8 focused on monitoring, logging, and operational visibility. Security controls are only effective when administrators can detect misuse, troubleshoot incidents, and validate system behavior.

CloudWatch logging was enabled during EKS cluster creation to capture critical control plane logs, including:

- API logs
- Audit logs
- Authenticator logs

These logs provided visibility into:

- Administrative actions
- Authentication attempts
- Kubernetes API requests
- Security-relevant operational events

In addition to CloudWatch, Kubernetes-native monitoring commands were used extensively:

- `kubectl get`
- `kubectl describe`
- `kubectl logs`

These tools provided direct insight into:

- Pod health
- Deployment status
- Secret references
- NetworkPolicy existence
- RBAC objects

This phase demonstrated:

- Security observability
- Incident awareness
- Operational troubleshooting
- Administrative accountability

Although advanced tools such as Prometheus, Grafana, and GuardDuty were not fully integrated, their roles were conceptually documented for future enhancement.

The key lesson from this phase was that visibility is essential to security. Without monitoring, organizations may deploy controls but remain blind to failures, misuse, or attack attempts.

AWS Console - IAM > IAM users > eks-project-admin

CloudShell

```
us-east-2 +
$ cd ~/secure-eks-project
secure-eks-project $ pwd
/home/cloudshell-user/secure-eks-project
secure-eks-project $ kubectl get secret app-secret -n secure-demo
NAME      TYPE      DATA   AGE
app-secret Opaque    2       30m
secure-eks-project $ kubectl describe secret app-secret -n secure-demo
Name:
Namespace: secure-demo
Labels:
Annotations:
Type: Opaque
Data:
---
DB_PASSWORD: 16 bytes
DB_USERNAME: 5 bytes
secure-eks-project $ kubectl describe deployment backend -n secure-demo
Name:
Namespace: secure-demo
CreationTimestamp: Sat, 12 May 2024 14:48:14 -0800
Labels:
Annotations: deployment.kubernetes.io/revision: 1
Selector: app=backend
Replicas: 1 desired | 1 updated | 1 total | 1 available | 0 unavailable
StrategyType: RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
Labels: app=backend, tier=backend
Containers:
  backend:
    Image: nginx
    Port: 80/TCP
    Host Port: 80/TCP
    Environment:
      DB_USERNAME: cset to the key 'DB_USERNAME' in secret 'app-secret': Optional: false
      DB_PASSWORD: cset to the key 'DB_PASSWORD' in secret 'app-secret': Optional: false
    Mounts:
    Volumes:
    Node-Selectors:
    Tolerations:
    Conditions:
      Type          Status Reason
      ----          -
      Available     True   MinimumReplicasAvailable
      Progressing    True   NewReplicaSetAvailable
OldReplicaSets:
NewReplicaSet: backend-948b4488rd (1/1 replicas created)
Events:
secure-eks-project $ kubectl exec -n secure-demo deployment/backend -- printenv | grep DB_
DB_USERNAME=admin
DB_PASSWORD=DummyPassword123
secure-eks-project $ cat app.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: secure-demo
---
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
  namespace: secure-demo
type: Opaque
stringData:
  DB_USERNAME: admin
  DB_PASSWORD: DummyPassword123
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: secure-demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
```

CloudShell Feedback Console Mobile App

© 2024, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

59°F Mostly cloudy

11:16 AM 5/2/2024

AWS Console - IAM > IAM users > eks-project-admin

CloudShell

```
us-east-2 +
$ cat app.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: secure-demo
---
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
  namespace: secure-demo
type: Opaque
stringData:
  DB_USERNAME: admin
  DB_PASSWORD: DummyPassword123
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: secure-demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
```

CloudShell Feedback Console Mobile App

© 2024, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

59°F Mostly cloudy

11:16 AM 5/2/2024

AWS Console - IAM > IAM users > eks-project-admin

CloudShell

us-east-2

```
name: app-secret
key: DB_PASSWORD

...
apiVersion: v1
kind: Service
secure-eks-project $ kubectl describe deployment frontend -n secure-demo
Name: frontend
Namespace: secure-demo
CreationTimestamp: Sat, 02 May 2026 14:48:48 +0000
Labels: <none>
Annotations: deployment.kubernetes.io/revision: 1
Selector: app=frontend
Replicas: 1 desired | 1 updated | 1 total | 1 available | 0 unavailable
StrategyType: RollingUpdate
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels: app=frontend
  Tolerations: tier=frontend
Containers:
  frontend:
    Image: nginx
    Port: 80/TCP
    Host Port: 80/TCP
    Environment: <none>
    Mounts: <none>
    Volumes: <none>
    Node-Selectors: <none>
    Tolerations: <none>
Conditions:
  Type Status Reason
  ----
  Available True MinimumReplicasAvailable
  Progressing True NewReplicaSetAvailable
  OldReplicaSets: <none>
  NewReplicaSet: frontend-fbb7fc699 (1/1 replicas created)
Events: <none>
secure-eks-project $ kubectl describe deployment backend -n secure-demo
Name: backend
Namespace: secure-demo
CreationTimestamp: Sat, 02 May 2026 14:48:48 +0000
Labels: <none>
Annotations: deployment.kubernetes.io/revision: 1
Selector: app=backend
Replicas: 1 desired | 1 updated | 1 total | 1 available | 0 unavailable
StrategyType: RollingUpdate
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels: app=backend
  Tolerations: tier=backend
Containers:
  backend:
    Image: nginx
    Port: 80/TCP
    Host Port: 80/TCP
    Environment:
      DB_USERNAME: <set to the key 'DB_USERNAME' in secret 'app-secret' Optional: false
      DB_PASSWORD: <set to the key 'DB_PASSWORD' in secret 'app-secret' Optional: false
    Mounts: <none>
    Volumes: <none>
    Node-Selectors: <none>
    Tolerations: <none>
Conditions:
  Type Status Reason
  ----
  Available True MinimumReplicasAvailable
  Progressing True NewReplicaSetAvailable
  OldReplicaSets: <none>
  NewReplicaSet: backend-848b488fd (1/1 replicas created)
Events: <none>
secure-eks-project $ kubectl get deployment frontend -n secure-demo -o yaml
bash: kubectl: command not found
secure-eks-project $ kubectl get deployment frontend -n secure-demo -o yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    deployment.kubernetes.io/revision: "1"
  kubectl.kubernetes.io/latest-applied-configuration: |
    {"apiVersion":"apps/v1","kind":"Deployment","metadata":{"annotations":{},"name":"frontend","namespace":"secure-demo"},"spec":{"replicas":1,"selector":{"matchLabels":{"app":"frontend"}},"template":{"metadata":{"labels":{"app":"frontend","tier":"frontend"},"spec":{"containers":[{"image":"nginx","name":"frontend","ports":[{"containerPort":80}]}]}}}}}
  creationTimestamp: "2026-05-02T14:48:48Z"
  generation: 1
  name: frontend
  namespace: secure-demo
  resourceVersion: "8454"
  uid: "4982f992-6811-4421-8123-4db385adb2f9"
  ...
```

AWS Console - IAM > IAM users > eks-project-admin

CloudShell

us-east-2

```
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    deployment.kubernetes.io/revision: 1
  selector: app=backend
  replicas: 1 desired | 1 updated | 1 total | 1 available | 0 unavailable
  strategyType: RollingUpdate
  rollingUpdateStrategy: 25% max unavailable, 25% max surge
  podTemplate:
    labels: app=backend
    tier=backend
    containers:
      backend:
        image: nginx
        port: 80/TCP
        hostPort: 80/TCP
        environment:
          DB_USERNAME: <set to the key 'DB_USERNAME' in secret 'app-secret' Optional: false
          DB_PASSWORD: <set to the key 'DB_PASSWORD' in secret 'app-secret' Optional: false
        mounts: <none>
        volumes: <none>
        nodeSelectors: <none>
        tolerations: <none>
    conditions:
      type status reason
      ----
      available true minimumReplicasAvailable
      progressing true newReplicaSetAvailable
      oldReplicaSets: <none>
      newReplicaSet: backend-848b488fd (1/1 replicas created)
  events: <none>
secure-eks-project $ kubectl get deployment frontend -n secure-demo -o yaml
bash: kubectl: command not found
secure-eks-project $ kubectl get deployment frontend -n secure-demo -o yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    deployment.kubernetes.io/revision: "1"
  kubectl.kubernetes.io/latest-applied-configuration: |
    {"apiVersion":"apps/v1","kind":"Deployment","metadata":{"annotations":{},"name":"frontend","namespace":"secure-demo"},"spec":{"replicas":1,"selector":{"matchLabels":{"app":"frontend"}},"template":{"metadata":{"labels":{"app":"frontend","tier":"frontend"},"spec":{"containers":[{"image":"nginx","name":"frontend","ports":[{"containerPort":80}]}]}}}}}
  creationTimestamp: "2026-05-02T14:48:48Z"
  generation: 1
  name: frontend
  namespace: secure-demo
  resourceVersion: "8454"
  uid: "4982f992-6811-4421-8123-4db385adb2f9"
  ...
```

AWS Console - Signup | Edit default payment preferences | eks-project-admin | IAM | Glue | Welcome to nginx | Ask Gemini | New Chrome available

us-east-1:console.aws.amazon.com/iam/home?region=us-east-2#/users/details/eks-project-admin?section=security_credentials

Search [Alt+S] Ask Amazon Q

IAM > IAM users > eks-project-admin

CloudShell

us-east-2

```

"enabled": true
},
{
  "types": [
    "controllerManager",
    "scheduler"
  ],
  "enabled": false
}
]
}
}

secure-eks-project $ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
ip-192-168-77-24-us-east-2-compute.internal Ready    <none>   4h13m v1.30.14-eks-4073768
secure-eks-project $ kubectl get pods -A
NAMESPACE    NAME                                READY    STATUS    RESTARTS   AGE
kube-system  aws-node-95xox                     2/2      Running   0           4h13m
kube-system  aws-node-gwtt1                     2/2      Running   0           4h13m
kube-system  coredns-7898d489fc-d916f          1/1      Running   0           4h17m
kube-system  coredns-7898d489fc-m8fw2          1/1      Running   0           4h17m
kube-system  kube-proxy-9f9dy                  1/1      Running   0           4h13m
kube-system  kube-proxy-zhg6f                  1/1      Running   0           4h13m
kube-system  metrics-server-59d4b7b6f-2gkxs    1/1      Running   0           4h11m
kube-system  metrics-server-59d4b7b6f-c3odp    1/1      Running   0           4h11m
secure-demo  attacker                           0/1      Completed 0           3h14m
secure-demo  backend-84db4488f6-tb82l          1/1      Running   0           3h25m
secure-demo  frontend-fbb7c699-zptvh           1/1      Running   0           3h25m

secure-eks-project $ kubectl logs deployment/frontend -n secure-demo
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/05/02 14:48:52 [notice] 181: using the "md" event method
2026/05/02 14:48:52 [notice] 181: nginx/1.29.8
2026/05/02 14:48:52 [notice] 181: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/05/02 14:48:52 [notice] 181: OS: Linux 6.1.166-197.309.amzn2023.x86_64
2026/05/02 14:48:52 [notice] 181: getrlimit(RLIMIT_NOFILE): 65536:1640876
2026/05/02 14:48:52 [notice] 181: start worker processes
2026/05/02 14:48:52 [notice] 181: start worker process 29
2026/05/02 14:48:52 [notice] 181: start worker process 30
192.168.77.24 - - [02/May/2026:14:51:23 +0000] "GET / HTTP/1.1" 200 896 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.8.0 Safari/537.36" "-"

```

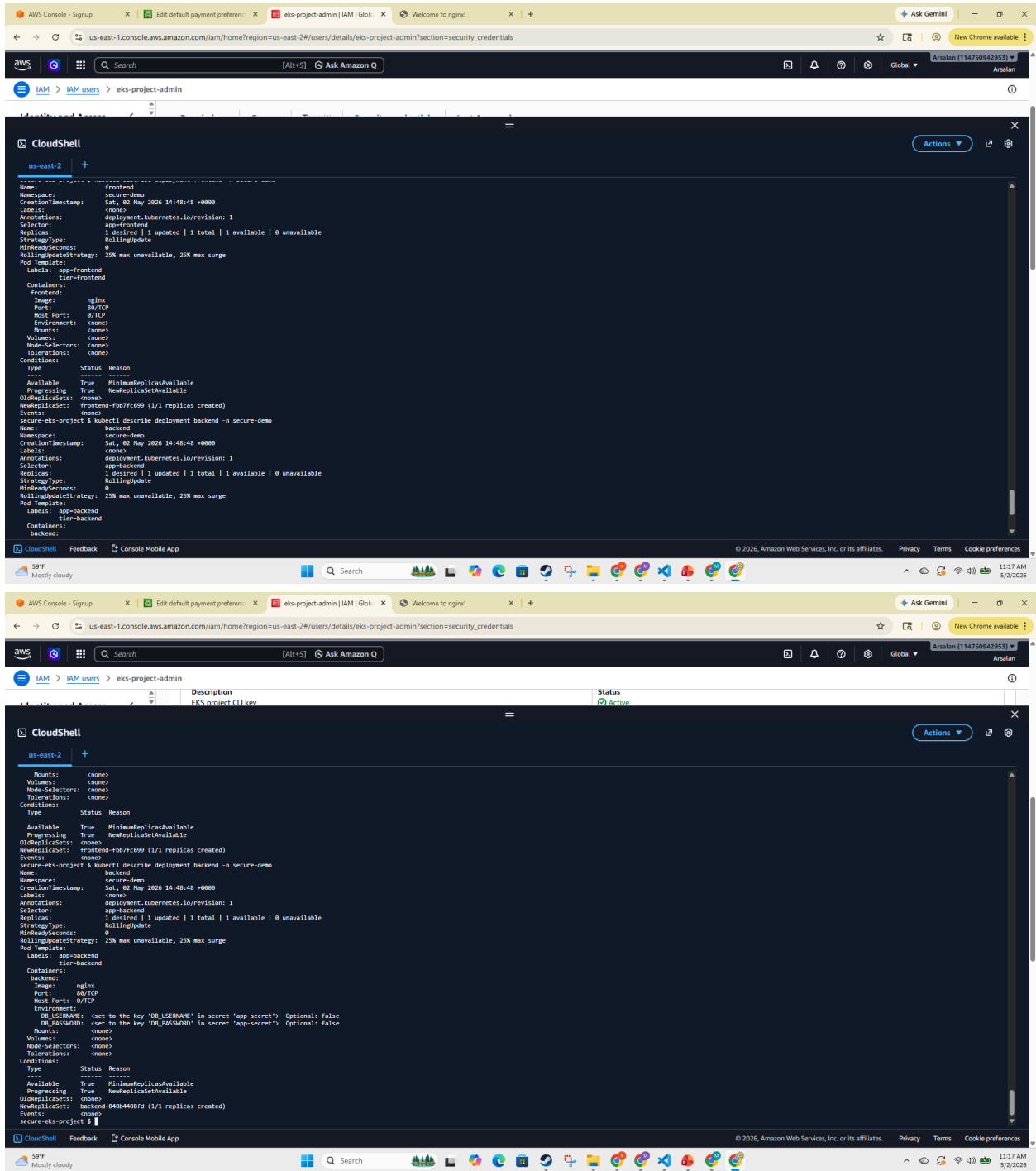
CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

55°F Mostly cloudy

Search

11:17 AM 5/2/2026



PHASE 9: THREAT SIMULATION & MITIGATION

Phase 9 represented the culmination of the project by simulating realistic security threats and demonstrating mitigation strategies.

Threat Scenario 1: Kubernetes RBAC Misconfiguration

A service account named student-user was created to simulate a limited internal user. Initially, excessive permissions were considered a risk because overprivileged users can modify or destroy workloads.

To mitigate this, a read-only Kubernetes Role and RoleBinding were applied within the secure-demo namespace.

Validation using kubectl auth can-i demonstrated:

- student-user could list pods
- student-user could not delete pods
- student-user could not create deployments

This successfully demonstrated least privilege and reduced Kubernetes attack surface.

Threat Scenario 2: AWS IAM Over-Permission

AdministratorAccess was initially used to simplify deployment. While operationally convenient, this represented a major cloud security risk because such permissions allow unrestricted infrastructure modification.

To address this, a least-privilege IAM policy was designed conceptually to limit future users to:

- EKS read-only
- EC2 describe-only
- CloudWatch monitoring-only

This scenario demonstrated:

- IAM privilege risk awareness
- Access reduction strategy
- Cloud governance maturity

AWS Console - Signup x Edit default payment preferenc... x eks-project-admin | IAM | Glo... x Welcome to rgind... x + Ask Gemini

us-east-1.console.aws.amazon.com/iam/home?region=us-east-2#/users/details/eks-project-admin?section=security_credentials

us-east-2 IAM IAM users eks-project-admin

Description FKS project CLI key Status Active

CloudShell us-east-2 + Actions

```
secure-eks-project $ pwd
/home/cloudshell-user/secure-eks-project
secure-eks-project $ nano rbac.yaml
secure-eks-project $ kubectl apply -f rbac.yaml
serviceaccount/student-user created
role.rbac.authorization.k8s.io/student-read-only-role created
rolebinding.rbac.authorization.k8s.io/student-read-only-binding created
secure-eks-project $ kubectl get serviceaccount -n secure-demo
NAME                AGE
default              0
student-user         0
secure-eks-project $ kubectl get role -n secure-demo
NAME                CREATED AT
student-read-only-role 2026-05-02T18:28:36Z
secure-eks-project $ kubectl get rolebinding -n secure-demo
NAME                ROLE              AGE
student-read-only-binding  role/student-read-only-role 33s
secure-eks-project $ kubectl auth can-i list pods --as=system:serviceaccount:secure-demo:student-user -n secure-demo
yes
secure-eks-project $ kubectl auth can-i delete pods --as=system:serviceaccount:secure-demo:student-user -n secure-demo
no
secure-eks-project $ kubectl auth can-i create deployments --as=system:serviceaccount:secure-demo:student-user -n secure-demo
no
secure-eks-project $
```

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

60°F Cloudy Search 11:30 AM 5/2/2026

AWS Console - IAM > IAM users > eks-project-admin

CloudShell

```
us-east-2 +
secure-eks-project $ kubectl get role -n secure-demo
NAME                                CREATED AT
student-read-only-role             2026-05-02T18:28:36Z
secure-eks-project $ kubectl get rolebinding -n secure-demo
NAME                                ROLE
student-read-only-binding          Role/student-read-only-role
secure-eks-project $ kubectl auth can-i list pods --as=system:serviceaccount:secure-demo:student-user -n secure-demo
yes
secure-eks-project $ kubectl auth can-i delete pods --as=system:serviceaccount:secure-demo:student-user -n secure-demo
no
secure-eks-project $ kubectl auth can-i create deployments --as=system:serviceaccount:secure-demo:student-user -n secure-demo
no
secure-eks-project $ aws sts get-caller-identity
{
  "UserId": "AIDARWQ5Q3U7YSMWQW4",
  "Account": "114750942953",
  "Arn": "arn:aws:iam:114750942953:user/eks-project-admin"
}
secure-eks-project $ aws eks list-clusters --region us-east-2
{
  "clusters": [
    "secure-eks-demo"
  ]
}
secure-eks-project $ nano least-privilege-policy.json
secure-eks-project $ cat least-privilege-policy.json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EKSShadowAccess",
      "Effect": "Allow",
      "Action": [
        "eks:DescribeCluster",
        "eks:ListClusters",
        "eks:ListNodegroups",
        "eks:DescribeNodegroup"
      ],
      "Resource": "*"
    },
    {
      "Sid": "CloudWatchShadowAccess",
      "Effect": "Allow",
      "Action": [
        "logs:DescribeLogGroups",
        "logs:DescribeLogStreams",
        "logs:GetLogEvents"
      ],
      "Resource": "*"
    }
  ]
}
```

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

60°F Cloudy 11:33 AM 5/2/2026

AWS Console - IAM > IAM users > eks-project-admin

CloudShell

```
us-east-2 +
"clusters": [
  "secure-eks-demo"
]
}
secure-eks-project $ nano least-privilege-policy.json
secure-eks-project $ cat least-privilege-policy.json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EKSShadowAccess",
      "Effect": "Allow",
      "Action": [
        "eks:DescribeCluster",
        "eks:ListClusters",
        "eks:ListNodegroups",
        "eks:DescribeNodegroup"
      ],
      "Resource": "*"
    },
    {
      "Sid": "CloudWatchShadowAccess",
      "Effect": "Allow",
      "Action": [
        "logs:DescribeLogGroups",
        "logs:DescribeLogStreams",
        "logs:GetLogEvents",
        "cloudwatch:ListMetricData",
        "cloudwatch:ListMetrics"
      ],
      "Resource": "*"
    },
    {
      "Sid": "EC2ShadowAccess",
      "Effect": "Allow",
      "Action": [
        "ec2:DescribeInstances",
        "ec2:DescribeSecurityGroups",
        "ec2:DescribeSubnets",
        "ec2:DescribeVpcs",
        "ec2:DescribeNetworkInterfaces"
      ],
      "Resource": "*"
    }
  ]
}
secure-eks-project $
```

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

60°F Cloudy 11:33 AM 5/2/2026